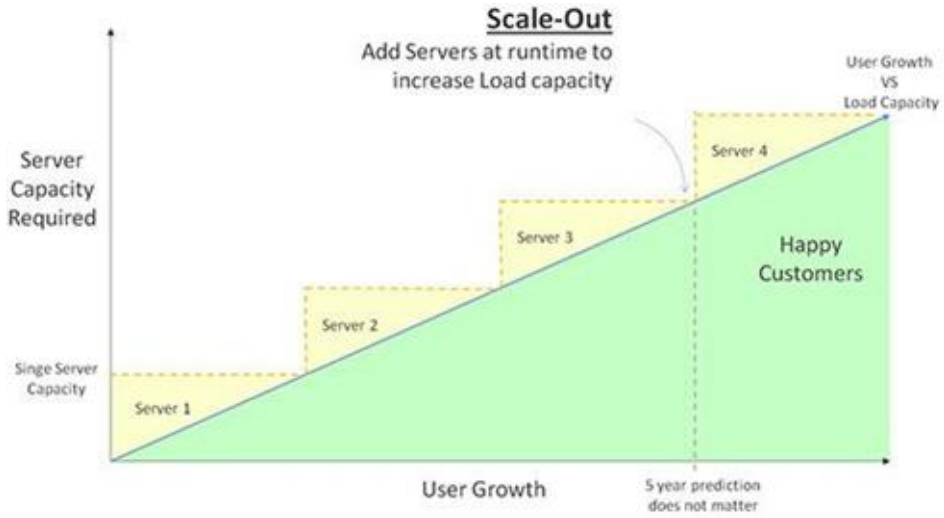
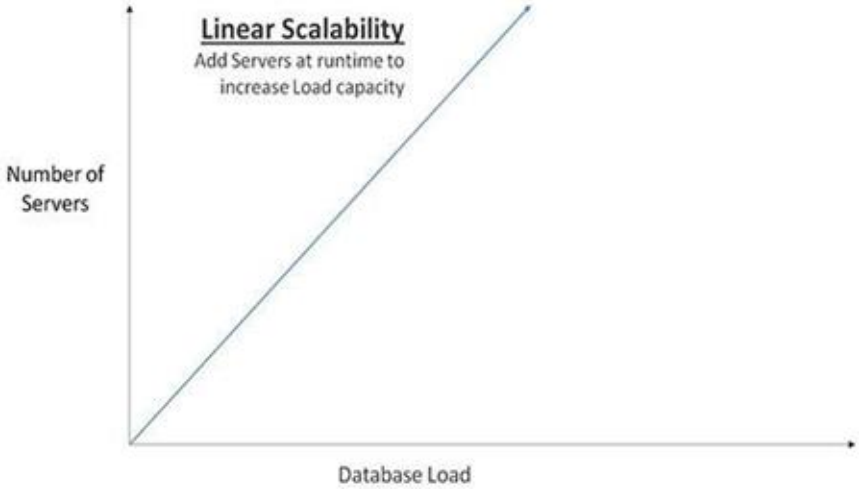
Last week we looked at NoSQL.

Let’s dig into it again in more detail.

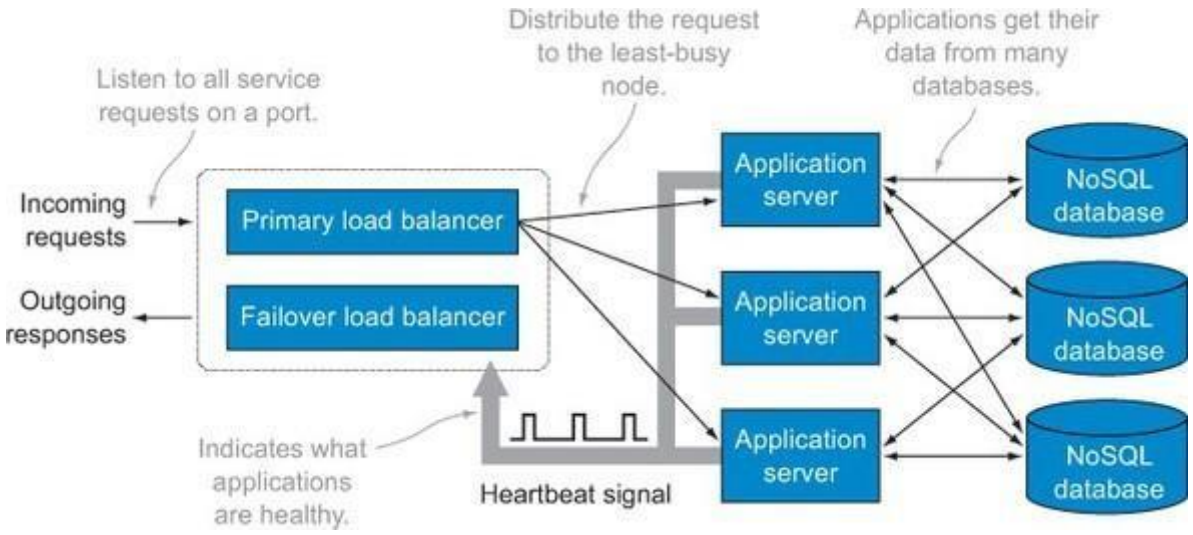
WHY NoSQL?

Distributed Mindset





High Availability



Flexible Data Models

USER Data

RDBMS Rows

ID	Name	Age	Status
47	John	25	Active
48	Andy	27	In-Active
49	Emma	22	Active

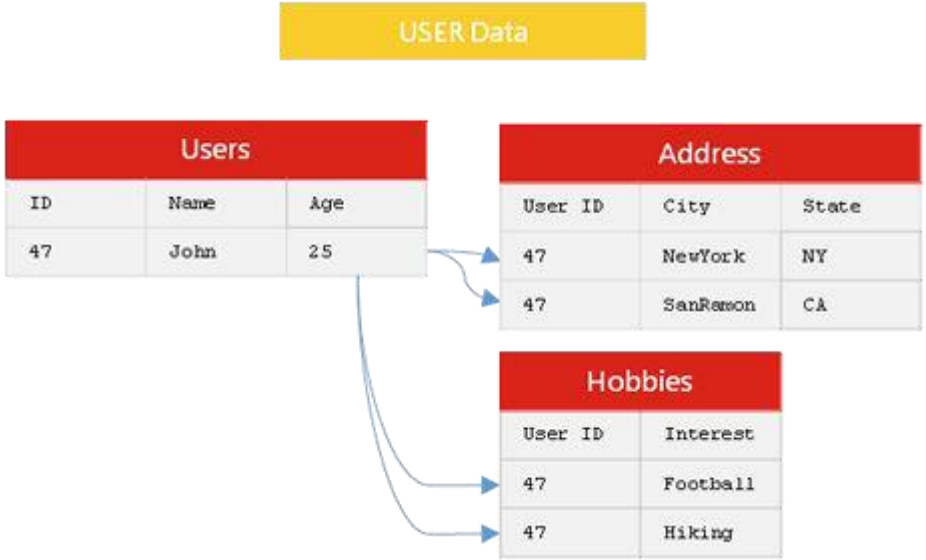
NoSQL Document

```
"key1" =
{
    "ID": 47,
    "Name": "John",
    "Age: 25,
    "Status": "Active"
}

"key2" =
{
    "ID": 48,
    "Name": "Andy",
    "Age: 27,
    "Status": "In-Active"
}


"key1" =
{
    "ID": 49,
    "Name": "Emma",
    "Age: 22,
    "Status": "Active"
}
```

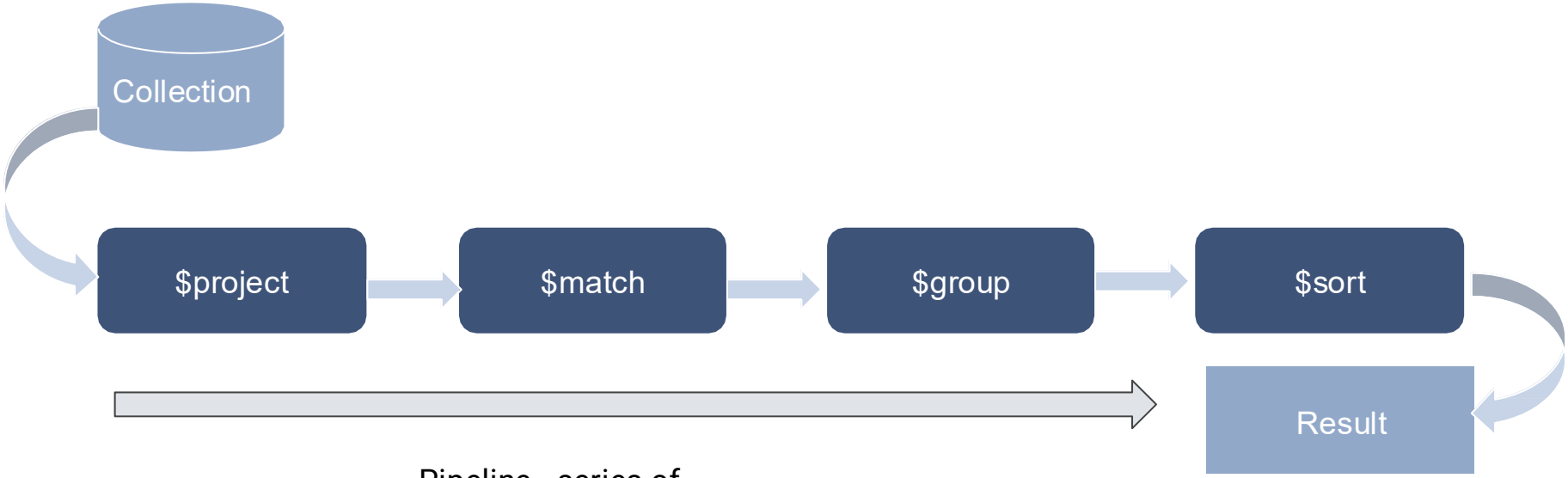
JSON



NoSQL Document

```
"key1" =
{
   "ID": 47,
   "Name": "John",
   "Age: 25
   "Hobbies": [ Football, Hiking ],
   "Address": [
                 {
                    "City": "NewYork",
                    "State": "NY"
                 },
                 {
                    "City": "SanRamon",
                    "State": "CA"
                 }
                ]
}
```

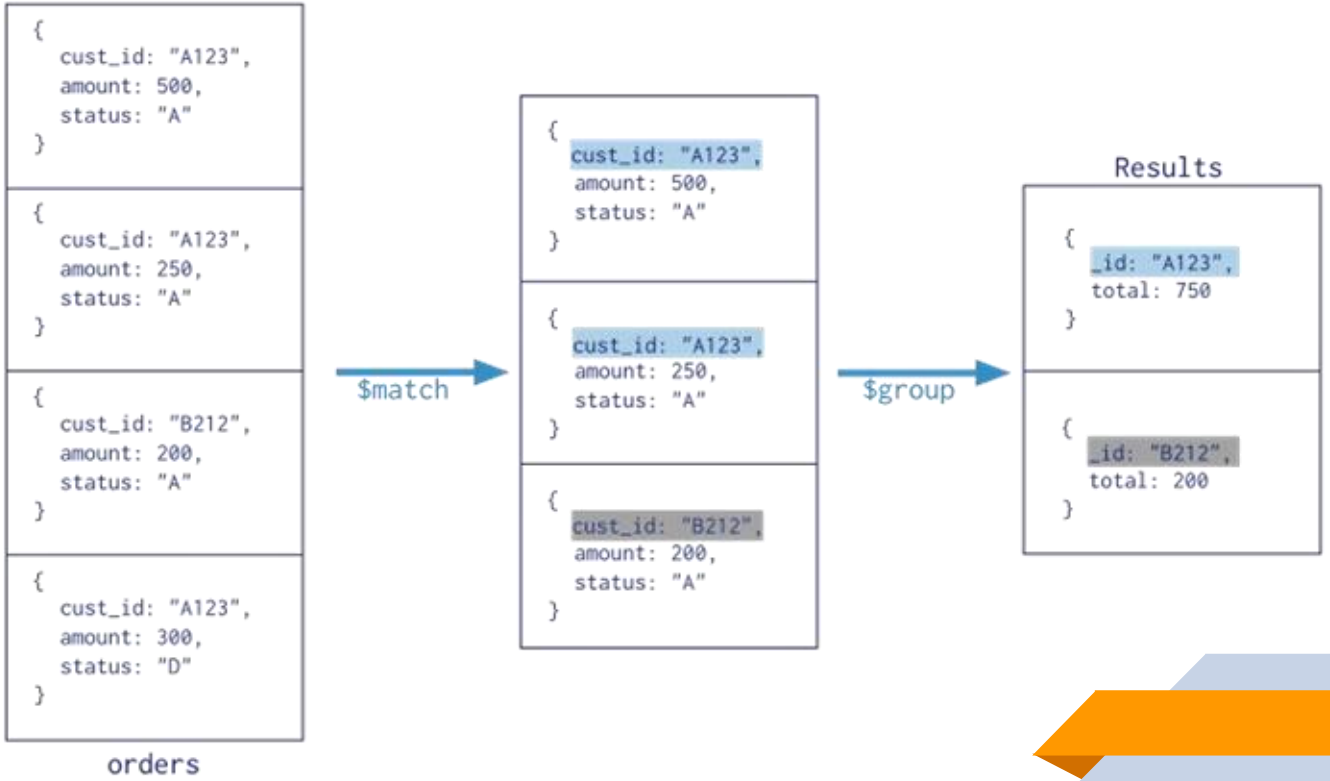
Aggregation Pipeline



Pipeline - series of stages

Collection

```
db.orders.aggregate( [
    $match stage →   { $match: { status: "A" } },
    $group stage →   { $group: { _id: "$cust_id",total: { $sum: "$amount" } } }
                   ] )
```



$match

- Filters the order documents to with a size of medium.
- Filters the order documents to those in a date range specified using $gte and $lt.
- Passes the remaining documents to the $group stage.

$group

- Groups the remaining documents by any field.
- Can use $sum to calculate the total for specific set of data. The total thus obtained is stored and returned by the aggregation pipeline.
- Groups the documents by date using $dateToString.
- For each group, calculates:
  a. Total order value using $sum and $multiply.
  b. Average order quantity using $avg.
- Passes the grouped documents to the $sort stage.

$sort

- Sorts the documents by the total order value for each group in descending order (-1).
- Returns the sorted documents.

For other operators:
https://docs.mongodb.com/manual/meta/aggregation-quick-reference/#_std-label-aggregation-expressions

Nested Documents

Example of Nested Document

```
{
  "address": {
     "building": "1007",
     "coord": [ -73.856077, 40.848447 ],
     "street": "Morris Park Ave",
     "zipcode": "10462"
  },
  "borough": "Bronx",
  "cuisine": "Bakery",
  "grades": [
     { "date": { "$date": 1393804800000 }, "grade": "A", "score": 2 },
     { "date": { "$date": 1378857600000 }, "grade": "A", "score": 6 },
     { "date": { "$date": 1358985600000 }, "grade": "A", "score": 10 },
     { "date": { "$date": 1322006400000 }, "grade": "A", "score": 9 },
     { "date": { "$date": 1299715200000 }, "grade": "B", "score": 14 }
  ],
  "name": "Morris Park Bake Shop",
  "restaurant_id": "30075445"
}
```

Data retrieval on Nested Document

Operators used to retrieve data on Nested Document:

find()

$elematch()

$filter

Notes: Usage of these operators will be discussed more in detail in Workshop

Pattern Matching

The format for regular expressions is:

```
{ <field>: /pattern/<options> }
```

Or can use the **$regex** command:

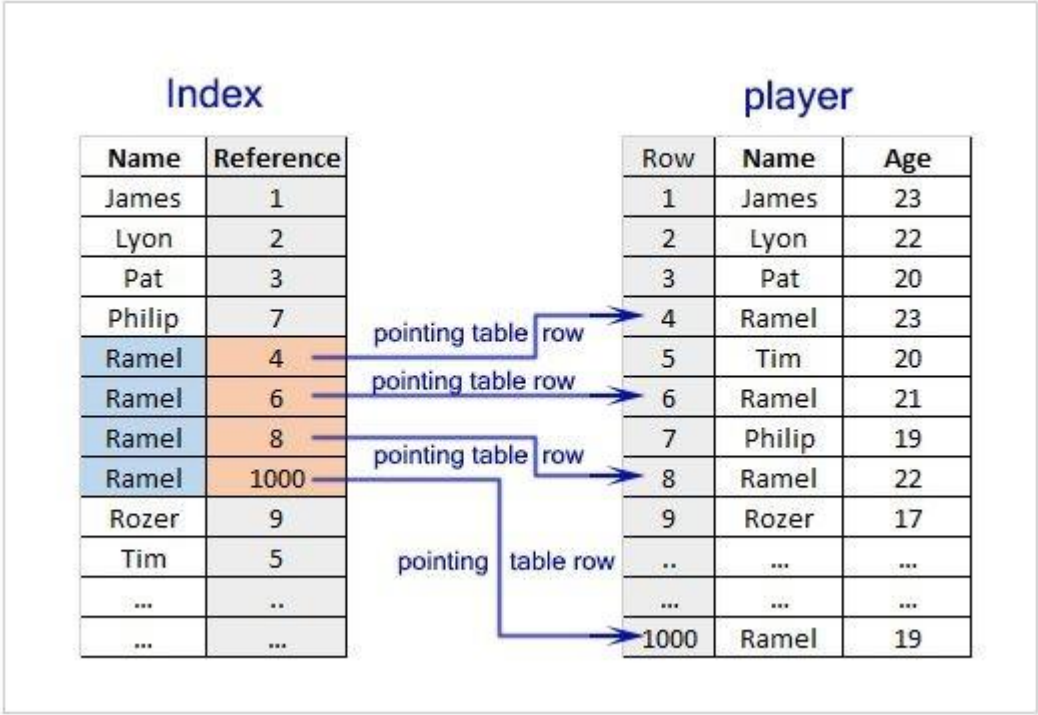
```
{ <field>: { $regex: /pattern/, $options: '<options>' } }
{ <field>: { $regex: 'pattern', $options: '<options>' } }
{ <field>: { $regex: /pattern/<options> } }
```

$regex

Few **$options** are available for using regular expressions such as:

Option	Description
i	Case insensitivity to match upper and lower cases
m	For patterns that include anchors (i.e. ^ for the start, $ for the end), match at the beginning or end of each line for strings with multiline values.
x	"Extended" capability to ignore all white space characters in the $regex pattern unless escaped or included in a character class.
s	Allows the dot character (i.e. .) to match all characters including newline characters.

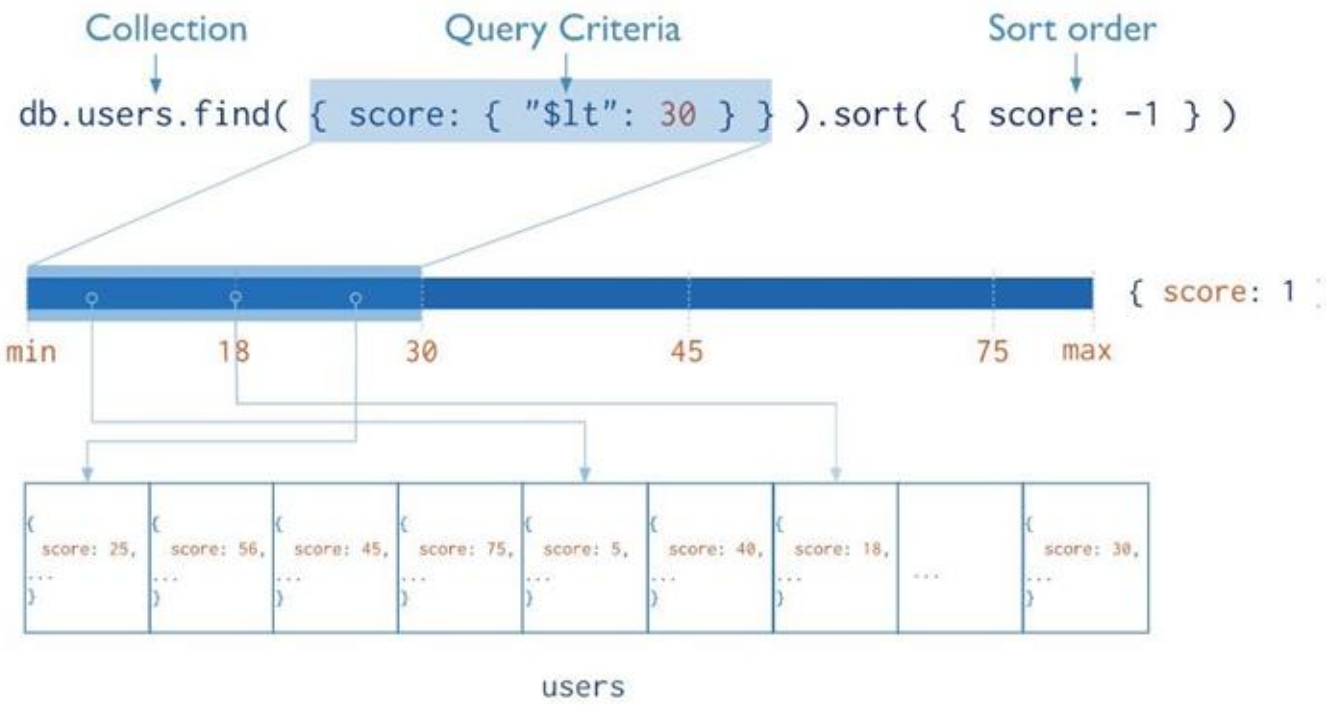
Indexes



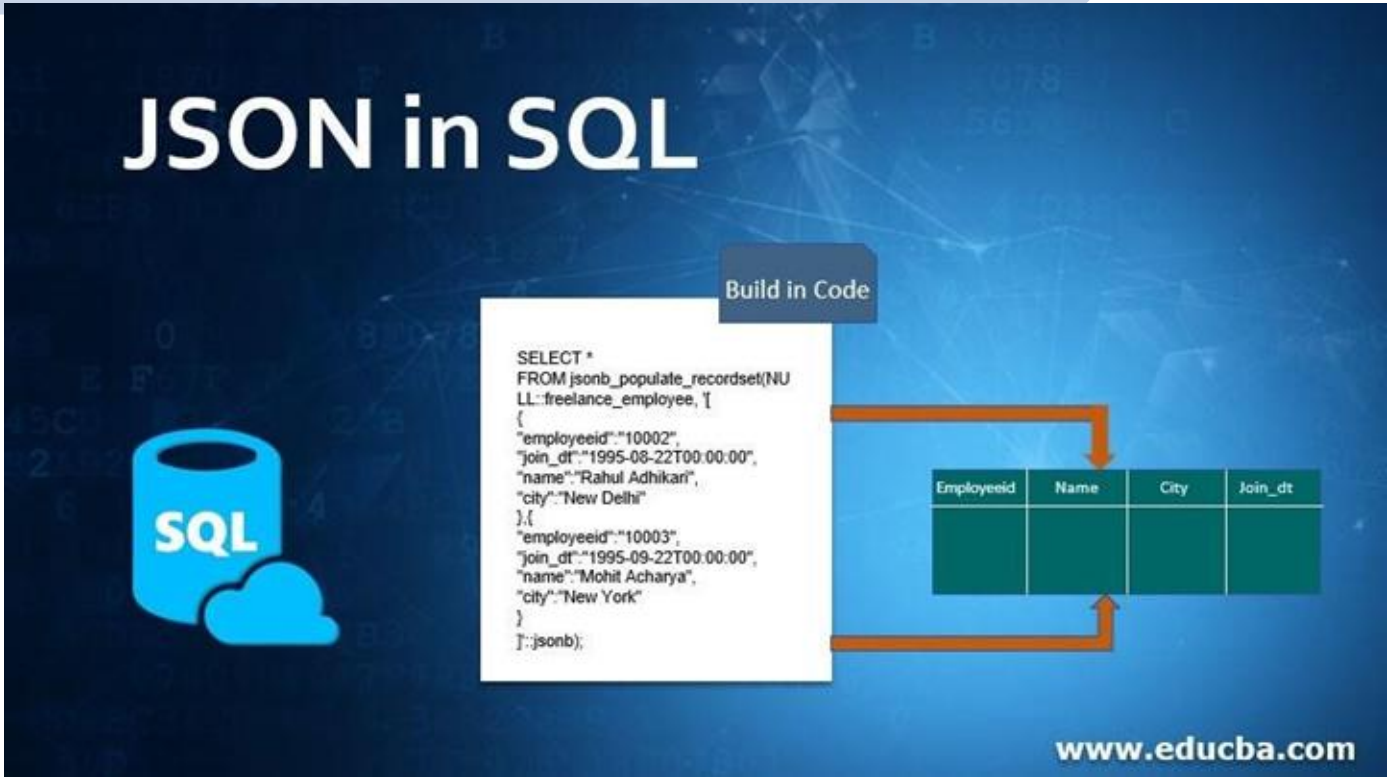
Name	Reference
James	1
Lyon	2
Pat	3
Philip	7
Ramel	4
Ramel	6
Ramel	8
Ramel	1000
Rozer	9
Tim	5
...	..
...	...

Row	Name	Age
1	James	23
2	Lyon	22
3	Pat	20
4	Ramel	23
5	Tim	20
6	Ramel	21
7	Philip	19
8	Ramel	22
9	Rozer	17
..	...	...
...	...	...
1000	Ramel	19

Index


Collection
Query Criteria
Sort order
db.users.find({ score: { "$lt": 30 } }).sort({ score: -1 })
{ score: 1
min
18
30
45
75
max
{ score: 25, ... }
{ score: 56, ... }
{ score: 45, ... }
{ score: 75, ... }
{ score: 5, ... }
{ score: 40, ... }
{ score: 18, ... }
...
{ score: 30, ... }
users

JSON and Relational Database



NoSQL VS’ Relational Database

	NoSQL	SQL
Model	Non-relational	Relational
	Stores data in JSON documents, key/value pairs, wide column stores, or graphs	Stores data in a table
Data	Offers flexibility as not every record needs to store the same properties	Great for solutions where every record has the same properties
	New properties can be added on the fly	Adding a new property may require altering schemas or backfilling data
	Relationships are often captured by denormalizing data and presenting all data for an object in a single record	Relationships are often captured in normalized model using joins to resolve references across tables
	Good for semi-structured, complex, or nested data	Good for structured data
Schema	Dynamic or flexible schemas	Strict schema
	Database is schema-agnostic and the schema is dictated by the application. This allows for agility and highly iterative development	Schema must be maintained and kept in sync between application and database
Transactions	ACID transaction support varies per solution	Supports ACID transactions
Consistency & Availability	Eventual to strong consistency supported, depending on solution	Strong consistency enforced
	Consistency, availability, and performance can be traded to meet the needs of the application (CAP theorem)	Consistency is prioritized over availability and performance
Performance	Performance can be maximized by reducing consistency, if needed	Insert and update performance is dependent upon how fast a write is committed, as strong consistency is enforced. Performance can be maximized by using scaling up available resources and using in-memory structures.
	All information about an entity is typically in a single record, so an update can happen in one operation	Information about an entity may be spread across many tables or rows, requiring many joins to complete an update or a query
Scale	Scaling is typically achieved horizontally with data partitioned to span servers	Scaling is typically achieved vertically with more server resources

THANKS!

Designed by : yamu.poudel@heraldcollege.edu.np